

Trabajo Integrador 2026

Seminario de Lenguajes Opción Python

El trabajo integrador consta del desarrollo de una aplicación de análisis y visualización de información de biodiversidad relacionada al estándar Darwin Core, a partir de este momento nos referiremos a la aplicación como la *app*. El desarrollo de este trabajo involucra el análisis de los datos, su limpieza y preparación necesaria y finaliza con el desarrollo de una interfaz gráfica. El objetivo es lograr aplicar sus conocimientos de programación mientras exploran datos reales y construyen una interfaz visual con Streamlit.

El proyecto se divide en dos etapas:

- **Primera Etapa:** se enfocarán en entender y sanitizar el conjunto de datos provenientes de <https://biodiversidad.ar/> . Tendrán que trabajar con conjuntos de datos relacionados al avistamiento de especies del reino animal y/o vegetal.
- **Segunda Etapa:** diseñarán la interfaz en Streamlit e integrarán los datos procesados. Se crearán diferentes secciones para mostrar información relevante sobre los datos del sistema.

El presente trabajo será realizado en grupos de **cinco** personas y se espera una distribución equitativa de tarea, sin embargo, todos los participantes deben tener una noción completa del sistema desarrollado.

Además de mejorar su manejo de Python y librerías como *Pandas*, *Matplotlib* y *Streamlit* (entre otras), este proyecto les permitirá desarrollar habilidades de trabajo en equipo, control de versiones con *Git* (*GitLab*) y documentación de código.

A continuación describe el trabajo a realizar y los requerimientos de la primera entrega.

Darwin Core

El estándar Darwin Core (DwC) es un conjunto de términos y definiciones diseñado para facilitar el intercambio de información sobre biodiversidad. Su propósito principal es estandarizar la forma en que se describen los registros biológicos —como la observación de una especie en un lugar y fecha determinados— permitiendo que datos provenientes de distintas instituciones puedan integrarse, compararse y analizarse de manera consistente. Darwin Core define campos comunes como nombre científico (*scientificName*), ubicación geográfica (*decimalLatitude*, *decimalLongitude*), fecha del evento (*eventDate*), institución proveedora (*institutionCode*) y tipo de registro (*basisOfRecord*), entre muchos otros. Al establecer un vocabulario compartido, reduce ambigüedades, mejora la interoperabilidad y favorece la reutilización de datos científicos.

Cada conjunto de datos es acompañado por un archivo de metadatos (información de la información), a modo de resumen:

- eml.xml: Descripción general del dataset, licencia, área de cobertura.
- meta.xml: Descripción de los archivos y formatos de los archivos del dataset:
 - tags "core" definen archivos con las columnas básicas del formato
 - tags "extensión" definen archivos con extensiones al formato -por ejemplo multimedia-
 - En los tags core y extensión un atributo "rowType" define el tipo de contenido del archivo:
 - Occurrence
 - Multimedia
 - DBADerivedData

Este estándar es promovido por la organización internacional *Biodiversity Information Standards (TDWG)* y es utilizado por redes y repositorios de datos de biodiversidad en todo el mundo, entre ellos *Global Biodiversity Information Facility (GBIF)*, que integra millones de registros provenientes de museos, universidades, herbarios y organismos gubernamentales de numerosos países. Instituciones científicas de los siete continentes publican sus datos bajo este estándar, lo que permite que la información sea accesible globalmente para investigación, conservación, toma de decisiones ambientales y desarrollo de políticas públicas basadas en evidencia.

Información de interés

A continuación se listan recursos para poder comprender mejor el estándar.

- Presentación del Portal de Datos de Biodiversidad de Argentina:

 [Portal de Datos de Biodiversidad de Argentina](#)

- Webinar de Darwincore:

 [Serie de webinars Calidad de datos de biodiversidad - Estándares, Darwin Core ...](#)

Antes de comenzar

Antes de comenzar con el análisis de una fuente de datos es fundamental -al menos- conocer las características generales de la fuente en cuestión. Debemos entender con qué

información trabajaremos, qué tipo de archivos, qué cantidad de archivos, qué tipos de datos, etc. Es inviable la idea de trabajar con un conjunto de datos de los cuales no tenemos conocimiento y pretender poder manipularlos con éxito. Por lo tanto, en este punto del TI se debe ya conocer las características de Darwincore, haber descargado al menos un conjunto de datos para simplemente ver su contenido y constatar que la teoría del estándar se condice con la realidad, y haber realizado cualquier otra actividad que crean necesaria para comprender la fuente de datos.

Exploración y documentación de los datasets

Objetivo de la sección

El objetivo de esta sección es realizar un primer acercamiento real a los conjuntos de datos de biodiversidad que se utilizarán durante el trabajo práctico.

Antes de comenzar a procesar información con Python, es fundamental comprender:

- ¿Qué información contienen los datasets?
- ¿Cómo están estructurados?
- ¿Qué tipo de datos incluyen?
- ¿Qué estándar utilizan para describir la biodiversidad?

En esta etapa no se requiere programar, sino analizar y comprender la estructura de los datos a partir de su documentación.

Descarga de datasets

Cada grupo deberá ingresar al siguiente link y descargar los datasets que allí se encuentren:

<https://archivos.linti.unlp.edu.ar/index.php/s/rPG30E5wUwy6Xbo>

Organización del repositorio

Se recomienda organizar el repositorio del proyecto con la siguiente estructura de directorios:

```
/
├── documentation
├── raw_datasets
└── processed_datasets
```

Importante: las carpetas `raw_datasets` y `processed_datasets` en el repositorio remoto **deben estar vacías** ya que son carpetas que solo contienen datasets y no lógica del sistema. A su vez si cada grupo sube el conjunto de datos con el que trabaja los servidores de la facultad no serán capaces de almacenar toda esta información.

Carpeta `documentation`

Contendrá la documentación generada durante el trabajo práctico.

En su mayoría los archivos deberán escribirse en **formato Markdown (.md)**, de manera que puedan visualizarse correctamente desde el repositorio en GitLab. Pero pueden añadirse diagramas e imágenes si corresponde.

Carpeta `raw_datasets`

Contendrá los **datasets originales descargados**, sin ningún tipo de procesamiento.

Es importante **no modificar estos archivos**, ya que representan la fuente de datos original.

Se recomienda renombrar las carpetas con los archivos de los datasets para trabajar con paths más sencillos.

Carpeta `processed_datasets`

Contendrá los **datasets generados o transformados durante el trabajo integrador**.

Ejercicio 1 - Documentación de los datasets

Ejercicio 1.A: Para cada uno de los datasets descargados se deberá generar un **archivo Markdown** que documente sus características principales.

El objetivo es producir una **documentación sintética pero efectiva**, que permita comprender rápidamente qué información contiene el dataset.

Se recomienda crear un archivo por dataset, por ejemplo:

- `documentation/dataset_a1.md`
- `documentation/dataset_a2.md`
- `documentation/dataset_a3.md`

Información general a documentar

Cada documento deberá incluir, como mínimo, la siguiente información:

- Nombre del dataset
- Institución proveedora
- Cantidad de registros
- Cobertura geográfica
- Cobertura temporal
- Separador de campos utilizado en los archivos
- Codificación de caracteres (encoding)
- Tipo de licencia
- Frecuencia de actualización
- Para cada archivo del dataset explicar su propósito.

Se recomienda comenzar con el análisis en este orden:

1. IADIZA
2. Xeno-canto
3. iNaturalist

Documentación de atributos del dataset

Ejercicio 1.B: Además de la información general, cada documento deberá describir los **atributos del dataset**. Para cada atributo explica en qué consiste y ejemplo de valores que puede tener.

Este paso es relevante para la etapa de validación posterior.

Lectura de datasets Darwin Core con Python

Objetivo de la sección

En la sección anterior se realizó un primer acercamiento conceptual a los datasets, analizando su documentación y estructura.

En esta sección comenzaremos a trabajar con Python para acceder a la información contenida en los archivos.

El objetivo es comprender cómo:

- abrir archivos de datos utilizando Python
- interpretar correctamente su estructura
- utilizar la librería estándar csv
- manipular registros de manera básica

Es importante destacar que en esta etapa **no se utilizarán librerías de análisis de datos como *pandas***, con el fin de comprender cómo funciona el procesamiento de datos en un nivel más básico.

Aclaración importante:

La siguiente aclaración aplica a todo el trabajo integrador: los datasets con los que trabajamos por cuestiones de practicidad son pequeños pero los reales con los que nos enfrentamos en el día a día profesional no lo son. No se deben cargar los mismos a memoria en su totalidad porque la misma no sería suficiente en entornos reales, deben utilizar estrategias que posibiliten el trabajo con datasets que ocupen más espacio que la memoria disponible (ver iteradores).

Estructura de un dataset Darwin Core

Como ya vimos los datasets basados en el estándar **Darwin Core Archive** suelen contener múltiples archivos. Entre ellos, es común encontrar:

- un archivo que contiene **los registros principales de biodiversidad**
- uno o más archivos con **información complementaria**

- un archivo de **metadata estructural** que describe cómo interpretar los datos

Por ejemplo, es habitual encontrar archivos como:

- `occurrence.txt`
- `meta.xml`
- `eml.xml`

Utilizando **Jupyter Notebook**, cada grupo deberá desarrollar código en Python que permita **abrir y explorar manualmente uno de los datasets descargados**.

Para ello deberán:

1. Identificar el archivo que contiene los **registros principales** del dataset.
2. Analizar el archivo de **metadata** (`meta.xml`) para obtener información necesaria para abrir correctamente el dataset.

En particular deberán identificar:

- el **separador de campos**
- la **codificación de caracteres**
- el **archivo que contiene los registros**

Con esta información deberán utilizar la librería estándar `csv` de Python para abrir el dataset correctamente.

En esta sección **la interpretación del archivo de metadata se realizará manualmente**, es decir, leyendo el archivo y extrayendo la información necesaria sin automatizar el proceso.

Una vez que hayan logrado abrir correctamente el dataset utilizando Python, deberán realizar las siguientes actividades.

Ejercicio 2 - Lectura del dataset

Para cada data set:

Ejercicio 2.A

Escribir una función que:

- Abra el archivo principal del dataset.

- utilice la librería `csv`.
- imprima las primeras **10 filas** del dataset.

Esto permitirá verificar que el archivo fue abierto correctamente.

Ejercicio 2.B

Escribir una función que retorne una lista completa de columnas del dataset.

Ejercicio 2.C

Escribir una función que retorne la posición (índice) de cada columna dentro del archivo.

Ejercicio 2.D

Escribir una función que retorne la cantidad total de registros del dataset.

Comparar este número con la cantidad de registros informada en la documentación del dataset.

Ejercicio 2.E

Escribir una función que retorne las columnas que posean al menos un dato con registro nulo.

Ejercicio 2.F

Escribir una función que retorne para cada columna el porcentaje de registros nulos.

Ejercicio 2.G

Escribir una función que, dado el nombre de una columna, retorne la cantidad de valores distintos presentes en dicha columna. Si la columna no existe se debe informar de dicha situación

Ejercicio 2.H

Escribir una función que, dado el nombre de una columna, retorne la frecuencia de aparición de cada valor.

El resultado puede representarse mediante un diccionario:

```
{  
    "Argentina": 1250,  
    "Brazil": 230,  
    "Chile": 80  
}
```

Ejercicio 2.I

Escribir una función que dada una columna y su tipo retorne lo establecido a continuación

Los tipos aceptados son:

- numeric: menor valor encontrado, mayor valor encontrado, promedio.
- coordinate: menor valor encontrado, mayor valor encontrado.
- text: cantidad de caracteres del texto más corto encontrado, cantidad de caracteres del texto más largo encontrado.

Ejercicio 2.J

Escribir una función que identifique columnas cuyo contenido sea completamente nulo en todos los registros.

Ejercicio 3 - Validación y calidad de datos

Objetivo de la sección

Los datasets provenientes del mundo real suelen contener **errores, inconsistencias o información incompleta**.

Antes de utilizar los datos para análisis o generación de nuevos datasets, es necesario realizar **procesos de validación y control de calidad**.

En esta sección se desarrollarán funciones que permitan:

- validar distintos tipos de datos
- detectar inconsistencias
- identificar registros problemáticos
- generar reportes de calidad del dataset

Los siguientes ejercicios deberán implementarse como **funciones reutilizables y aplicarse sobre los datasets que sea posible**.

Ejercicio 3.A

Los campos `decimalLatitude|latitudeDecimal`, `decimalLongitude|longitudeDecimal` deben representar coordenadas geográficas válidas.

Escribir una función que detecte registros donde:

- `decimalLatitude` no esté en el rango **`[-90, 90]`**
- `decimalLongitude` no esté en el rango **`[-180, 180]`**
- alguno de los valores no pueda convertirse a número

La función debe retornar:

- cantidad de registros inválidos
- listado de registros incorrectos.

Ejercicio 3.B

Un registro geográfico debería tener **ambas coordenadas**. Escribir una función que detecte registros donde:

- exista `decimalLatitude` pero no `decimalLongitude`
- exista `decimalLongitude` pero no `decimalLatitude`.

Ejercicio 3.C

Escribir una función que detecte:

- fechas con formato inválido
- fechas que no puedan interpretarse como fecha
- fechas posteriores al año actual

Puede consultar con su ayudante cómo obtener el formato válido de una fecha. Aplicar esta función de validación a todas los atributos posibles.

Ejercicio 3.D

Utilizando los "ID" escribir una función que detecte posibles **registros duplicados**.

La función deberá indicar:

- cuántos registros duplicados existen
- qué identificador se repite.

Ejercicio 3.E

El campo `countryCode` puede tomar solo un conjunto de valores específicos, investigar cuáles son y desarrollar una función que detecte valores no permitidos.

Ejercicio 3.F

El campo `coordinateUncertaintyInMeters` representa la incertidumbre en la definición de la coordenada.

Escribir una función que detecte registros donde:

- el valor no sea numérico
- el valor sea negativo

- el valor sea excesivamente grande (por ejemplo > 100).

Ejercicio 3.G

Escribir una función que genere un **resumen de calidad del dataset**, incluyendo:

- cantidad total de registros
- registros con coordenadas inválidas
- registros con fechas inválidas
- registros duplicados
- registros con información taxonómica incompleta

El resultado puede ser un diccionario y un pequeño reporte en texto.

Ejercicio 3.H

Establezca cotas de coordenadas para el américa del sur, es decir latitudes y longitudes máximas y mínimas que delimitan ocurrencias dentro del territorio (aproximado). A partir de estas cotas seteadas como constantes escriba una función que valide los campos de latitud y longitud de cada dataset.

Ejercicio 3.I

Escriba una función que agrupe y ejecute todas las validaciones para el campo latitud, y otra independiente para el campo longitud.

Ejercicio 4 - Inserción de registros en datasets

Objetivo de la sección

Hasta el momento se ha trabajado con la lectura y exploración de datasets. En esta sección se incorporará una nueva capacidad fundamental: la escritura y modificación de datos. Los objetivos son:

- crear nuevos registros compatibles con el dataset
- validar su estructura
- insertarlos correctamente en los archivos existentes
- generar nuevos datasets a partir de los originales

Esto permitirá comprender cómo se construyen y mantienen datasets en escenarios reales.

Consideraciones iniciales

Los datasets Darwin Core suelen estar compuestos por archivos tabulares (por ejemplo, `occurrence.txt`).

Para insertar nuevos registros correctamente, es necesario:

- respetar el **orden de las columnas**.
- utilizar el **mismo separador de campos**.
- mantener la **misma codificación**.
- asegurar consistencia con el resto de los datos.

Toda operación de inserción deberá generar un nuevo dataset en *processed_datasets* y en caso de existir actualizar el existente

Ejercicio 4.A

Definir una estructura (tupla, lista, diccionario, clase) en Python que represente un registro para cada uno de los datasets. No considerar el ID.

Ejercicio 4.B

Escribir una función que:

- reciba la lista de columnas del dataset
- genere un **registro vacío**, donde todos los valores estén inicializados (por ejemplo, con `None`, cadena vacía, 0, etc.)

Esto facilitará la creación de nuevos registros válidos. No considerar el ID.

Ejercicio 4.C

Escribir una función que valide un registro antes de insertarlo. Debe reutilizar todas las funciones posibles ya desarrolladas para este fin. No considerar el ID.

Ejercicio 4.D

Escribir una función que reciba la estructura definida en el inciso 4.A y retorne una lista/diccionario listo para ser añadido al csv que contiene el dataset.

Deberá considerar añadir el/los IDs a la información deduciendo cómo se construyen de acuerdo a cada dataset (autoincremental, random, etc).

Ejercicio 4.F

Escribir una función que:

1. Lea el dataset original.
2. Agregue un nuevo registro al final con datos ingresados por teclado.
3. escriba un nuevo archivo en `processed_datasets`.

El archivo resultante debe:

- conservar la estructura original
- incluir el nuevo registro correctamente formateado

Ejercicio 4.G

Extender la función anterior para permitir insertar múltiples registros en una sola operación.

Ejercicio 5 - Actualización de registros en datasets

Objetivo de la sección

En esta sección se abordará la modificación de registros existentes dentro de un dataset. Los objetivos son:

- identificar registros específicos dentro del dataset
- modificar uno o más de sus campos
- generar una nueva versión del dataset con los cambios aplicados

Este tipo de operaciones es fundamental cuando los datos:

- contienen errores
- requieren actualización
- deben ser enriquecidos con nueva información

Consideraciones iniciales

- No se deben modificar los archivos originales en `raw_datasets`.
- Toda actualización deberá generar un nuevo archivo en: `processed_datasets`

Ejercicio 5.A

Escribir una función que permite buscar registros por múltiples columnas. Deberá recibir el conjunto de columnas a filtrar con los valores deseados.

Ejemplos:

- por `occurrenceID (1234)`
- por `scientificName (Elaenia chilensis)` y `Country (Argentina)`
- por `Country (Argentina)`

La función debe retornar:

- los registros que cumplen la condición (todos sus campos).

Ejercicio 5.B

Escribir una función que:

- reciba un identificador de registro (por ejemplo `occurrenceID`)
- reciba el nombre de una columna
- reciba un nuevo valor

y permita actualizar dicho campo en el registro correspondiente.

Ejercicio 5.C

A partir de la función anterior desarrollar una nueva para permitir actualizar múltiples campos de un mismo registro en una sola operación

Ejemplo:

```
{  
  "country": "Argentina",  
  "stateProvince": "Buenos Aires"  
}
```

Ejercicio 5.D

Actualizar las funciones desarrolladas anteriormente para que antes de aplicar cambios, validar que:

- los nuevos valores sean correctos (reutilizar funciones de validación)
- no se introduzcan inconsistencias

Si la validación falla:

- el registro no debe modificarse
- se deben reportar los errores

Ejercicio 6 - Eliminación de registros en datasets

Objetivo de la sección

En esta sección se abordará la eliminación de registros dentro de un dataset.

El objetivo es que los estudiantes desarrollen herramientas para:

- identificar registros que deben ser eliminados
- aplicar criterios de filtrado
- generar un nuevo dataset sin los registros seleccionados

Este proceso es común en tareas de:

- limpieza de datos
- eliminación de outliers
- filtrado por condiciones específicas

Consideraciones

- No se deben modificar los archivos originales.
- Los resultados deben guardarse en `processed_datasets`

Ejercicio 6.A

Escribir una función que elimine un registro dado su identificador (`occurrenceID`, por ejemplo). En caso de no encontrarlo informe del error.

Ejercicio 6.B

Escribir una función que elimine registros cuyo valor en una columna pertenezca a una lista de valores suministrada por el usuario.

Ejemplo:

```
"Argentina", "Chile"
```

Para ello la función deberá recibir la columna de interés y el conjunto de valores a eliminar.

Ejercicio 6.C

Escribir una función que elimine registros que cumplan una condición, para eso la función recibirá:

- columnas de interés.
- condición: !=, ==, >, >=, <, <=.
- valor.

Ejercicio 6.E

Escribir una que sea capaz de *sanitizar* un determinado dataset de forma automática, para ello deberá recibir el nombre del dataset a sanitizar. Luego correrá todas las validaciones desarrolladas en la sección de validación y eliminará los datos conflictivos.

Se espera que por cada campo se encuentren claramente definidas las validaciones a aplicar, cada grupo definirá la estrategia que considere pertinente para este fin.

En resumen, este proceso debe:

1. lea el dataset original
2. elimine los registros correspondientes
3. genere un nuevo archivo en `processed_datasets`

Ejercicio 6.F

Evolucionar la función anterior para que además informe:

- cantidad de registros eliminados
- porcentaje sobre el total
- motivo de eliminación

Ejercicio 7 - Registro de operaciones

Objetivo de la sección

En las secciones anteriores se desarrollaron herramientas para:

- insertar nuevos registros
- actualizar información existente
- eliminar registros

En esta sección se incorporará un nuevo concepto fundamental: el registro de operaciones (logging).

El objetivo es que cada modificación realizada sobre un dataset quede registrada, permitiendo:

- conocer qué cambios se realizaron.
- cuándo se realizaron.
- sobre qué dataset.
- con qué tipo de operación.

Consideraciones iniciales

- Se deberá generar un archivo de log dentro del repositorio, por ejemplo: `logs/operations.log`
- Cada operación realizada sobre un dataset deberá registrarse en este archivo.
- Formato del log:
 - fecha y hora de la operación
 - nombre del dataset
 - tipo de operación (`INSERT`, `UPDATE`, `DELETE`)
 - cantidad de registros afectados

Ejemplo:

```
2026-03-23 21:35:10 | dataset_a1 | INSERT | 3 registros
```

Ejercicio 7.A

Escribir una función que retorne la fecha y hora actual en un formato legible.

Sugerencia: utilizar la librería estándar `datetime`.

Ejercicio 7.B

Escribir una función que:

- reciba el registro de cambio de una operación.
- los escriba en el archivo `operations.log`

La función debe:

- crear el archivo si no existe
- agregar nuevas líneas sin sobrescribir el contenido previo.

Ejercicio 7.C

Modificar las funciones de inserción para que:

- registren en el log cada operación realizada

Ejercicio 7.D

Modificar las funciones de actualización para que:

- registren cada modificación realizada
- indiquen la cantidad de registros afectados

Ejercicio 7.E

Modificar las funciones de eliminación para que:

- registren la operación.
- Indique cuántos registros fueron eliminados.

Ejercicio 7.F

Modificar las funciones para que:

- en caso de error (por ejemplo, validación fallida)
- también se registre el evento en el log

Ejemplo:

```
2026-03-23 21:42:10 | dataset_a1 | INSERT | 0 registros | ERROR
```

Ejercicio 8 - Inicialización de aplicación con Streamlit

Objetivo de la sección

En esta sección se dará inicio al desarrollo de una aplicación interactiva utilizando Streamlit, que será utilizada en la segunda parte del trabajo integrador.

El objetivo es:

- definir la estructura base de la aplicación
- organizar las distintas páginas
- comenzar a visualizar información generada en las secciones anteriores

En esta etapa no se espera implementar funcionalidades complejas, sino establecer una base sólida sobre la cual se trabajará posteriormente.

8.A Estructura de la aplicación

Se solicita crear una aplicación en Streamlit que contenga un menú lateral (**sidebar**) con las siguientes páginas:

- (P1) Inicio
- (P2) Estado del sistema
- (P3) Búsqueda (sin implementar)
- (P4) Visualización (sin implementar)

Las páginas (P3) y (P4) podrán quedar como placeholders sin funcionalidad en esta etapa.

8.B Página 1 – Inicio (P1)

La página de inicio deberá contener:

- Un título con el nombre de la aplicación (a elección).
- Un breve texto descriptivo que incluya:
 - propósito de la aplicación
 - importancia del análisis de datos de biodiversidad
- Una explicación breve del estándar **Darwin Core**:
 - qué es

- para qué se utiliza
- Instrucciones básicas de uso de la aplicación:
 - cómo navegar entre páginas
 - qué tipo de información se podrá consultar

8.C Página 2 – Estado del sistema (P2)

Esta página deberá mostrar (para esta primera parte) los logs generados. Para ello debe leer el archivo:

```
logs/operations.log
```

y mostrar su contenido de forma tabular en la aplicación.

Se espera:

- mostrar las operaciones realizadas
- permitir visualizar claramente:
 - fecha
 - dataset
 - operación
 - cantidad de registros
 - estado (si corresponde)

Consideraciones generales de implementación

El software implementado deberá funcionar correctamente tanto en Windows, Linux o Mac. Se deberá armar una estructura de directorios organizando los archivos en carpetas y subcarpetas de manera tal que mantengan el código organizado haciendo que sea fácil de actualizar y de corregir. Se evaluará tanto el código como su organización.

El código se deberá subir a un repositorio de GitLab designado por la cátedra. En este repositorio, cada equipo deberá incluir un archivo denominado **README.md** que contenga el nombre de los integrantes del equipo y las instrucciones para ejecutar cada aplicación. Además, se solicitará un archivo que enumere las bibliotecas necesarias, siguiendo la metodología explicada en las prácticas para este propósito.

No se permite subir ningún tipo de dataset al repositorio, son carpetas cuyo contenido deberá ser ignorado.

Para el desarrollo del código se deberá cumplir con los siguientes requerimientos:

- Los archivos entregables del ejercicio 1 serán varios archivos Markdown.
- Para los ejercicios 2 a 7, se deberá utilizar **Jupyter Notebooks** como entorno de ejecución y presentación de resultados. La implementación de la lógica y las funcionalidades requeridas deberá realizarse en archivos de Python (**.py**), los cuales deberán ser importados en los notebooks correspondientes. Se exige que el código se encuentre organizado siguiendo criterios de modularización, agrupando en cada archivo **.py** funciones relacionadas y manteniendo una estructura clara y coherente.
- El ejercicio 8 estará compuesto por un archivo principal (**.py**) que permitirá ejecutar la aplicación Streamlit y archivos (**.py**) individuales con cada página siguiendo las convenciones de Streamlit.
- El archivo README.md deberá tener indicaciones claras de cómo ejecutar el código de cada ejercicio, incluyendo creación del virtualenv, instalación de dependencias y comandos para ejecutar los ejercicios (al menos un ejemplo de cómo ejecutar uno de los notebooks y un ejemplo de cómo ejecutar la aplicación Streamlit).
- En esta primera etapa del trabajo integrador **NO se pueden utilizar librerías externas para leer los archivos como por ejemplo pandas o pydwtcase. Se deberán usar las librerías provistas con el intérprete como csv y json.**
- Usar la herramienta **Streamlit** para el desarrollo de la interfaz gráfica.
- Documentar el código usando docstrings en funciones y clases (estas últimas caso de ser utilizadas).
- Incluir un archivo **LICENSE** con la licencia del código.
- Se debe tener en cuenta las [guías de estilo de Python](#) para la escritura de código.
- Definir una estructura de carpetas que permita estructurar el código de forma prolija.

Consideraciones de la entrega y defensa

Si bien el trabajo es grupal, **la nota de la defensa y del progreso/desempeño semanal es INDIVIDUAL.**

La defensa se lleva a cabo durante el horario de consulta la semana posterior a la fecha de entrega. Este proceso implica la realización de un encuentro virtual o presencial con el ayudante asignado, durante el cual se formulan una serie de preguntas relacionadas con la entrega. Dichas preguntas están dirigidas a los distintos miembros del grupo, distribuyendo equitativamente la participación entre todos ellos. La finalidad es que cada integrante tenga la oportunidad de responder. La evaluación se basa en el desempeño de las respuestas, lo que determina finalmente la nota asignada a cada miembro del grupo en relación con la entrega realizada.

Esto implica que, aunque se trate de una entrega grupal, las calificaciones entre los miembros del grupo pueden variar.

Tener en cuenta que es fundamental mantener una participación activa tanto en las consultas prácticas como en el repositorio de GitLab. Esto permitirá al ayudante obtener una comprensión conceptual del conocimiento de cada participante del grupo.

Detalles de entrega

La fecha de entrega de la primera etapa es el día **4 de mayo a las 23:59 hs.** **La defensa será esta misma semana en horario de la consulta práctica.**

La primera entrega otorga al estudiante un **máximo de 35 puntos**, los cuales se basan en su desempeño durante las consultas, la defensa y participación en el repositorio.

Criterios de Evaluación

La distribución de los puntos de la entrega estará disponible en la rúbrica asociada a la tarea. Se evaluará:

- Funcionalidad implementada de acuerdo al enunciado.
- Cumplimiento de las consignas planteadas.
- Código subido en tiempo y forma al repositorio de GitLab indicado.
- Participación individual activa durante el desarrollo del trabajo, que incluye asistencia a las consultas (tanto virtuales como presenciales) y contribuciones al repositorio de GitLab.
- En la defensa, se espera que cada integrante del grupo demuestre los conocimientos utilizados para la realización del trabajo.